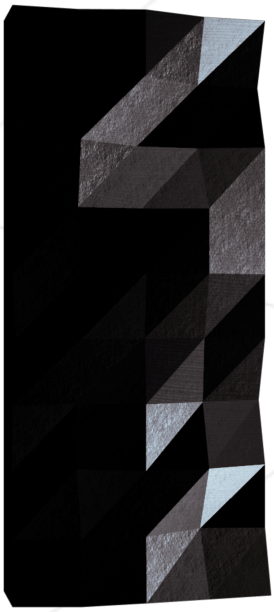

Avaj Launcher

Enter the Java path

Summary: This project is an introduction to the world of **`Java`** *at 42.*

Version: 5.2

Contents

I Introduction **2**

II Objectives **4**

III General instructions **6**

IV Mandatory part **7**

IV.1 Program behaviour 7
IV.2 Scenario file 8
IV.3 Weather generation 8
IV.4 Aircraft 8
IV.5 Simulation 9
IV.6 Validation 9

V Bonus part **10**

VI Submission and peer-evaluation **11**

Chapter I

Introduction

This is the first project in a series of four projects focused on `Java`. We are 42's counterpart, located in Cluj-Napoca, Romania, and the first implementation of the 42 program outside France. We started in 2014, and each year the number of applicants grows, as does the number of students we can accommodate. Feel free to visit us whenever you feel like traveling (ping me if you need info about cheap flights).
To create great software, writing code is not enough; you also need to design it first. This project introduces you to UML class diagrams and object-oriented design patterns, all implemented in `Java`. Writing `Java` code is easy since it is very similar to `C`, but the real challenge is writing good OO code in `Java`.

You will implement a minimal aircraft simulation program based on a given UML class diagram. `Unified Modeling Language` is used in software engineering to visualize the design of an application. It gives programmers a standardized way of thinking about object-oriented software before writing any line of code.

`Thinking in Java`

Chapter II

Objectives

Frankfurt Airport recently discovered that, due to frequent weather changes, it faces bottlenecks on some landing runways. To find a solution, it first needs to know which scenarios create the worst bottlenecks. The team decided to use a simulator to configure and analyze multiple scenarios, hoping this will highlight the root cause.

So they reached out to their top local software company and assigned this task. Here, the chief designer works on the concept and, after analyzing all the facets of the software, makes design decisions that he then passes on to you so you can build the simulator.

Since the software will run on multiple operating systems in a very strict enterprise environment, he decides to use a classic object-oriented language: Java.

He will provide you with:

- the UML class diagram
- the required object-oriented design patterns

What you need to know to be part of the team that develops the simulator:

- Interpreting class diagrams - this is how the architect communicates with you
- Observer, Singleton, and Factory design patterns - he knows this will not be the final version of the simulator and plans to extend it to address other needs the airport may have
- The basic syntax of `Java` and some of the core features of the language - this is obvious, since this is the language agreed upon

Object Oriented Design and design patterns are topics that each cover thousands of pages, so feel free to explore this domain and you will discover a new way of thinking about software engineering. And who knows? Maybe you will take that architect's role one day. Only a good implementation will be accepted, since this is the top software company in the city. For this to happen, it must have a clean design, be easy for your peers to read and understand, and be easy to change if the requirements are modified.

Gang of Four

Even though the mirage of a powerful IDE can be tempting, we strongly advise you to work with a good text editor. At the beginning, it is important to understand the internal workings of Java, and an IDE will hide them from you.

Chapter III

General instructions

- You are allowed to use language features up to the latest Java LTS version.
- You are not allowed to use any external libraries, build tools, or code generators.
- Do not use the default package.
- Create your own relevant packages following the `Java` package naming conventions.
- Java is compiled into an intermediate language, which will generate some .class files. Do not commit them to your repository!
- Make sure you have javac and java available as commands in your terminal.
- Compile the project by running the commands below in the root of your project folder.

```
$find * -name "*.java" > sources.txt
$javac @sources.txt
```

Chapter IV

Mandatory part

You need to implement an aircraft simulation program based on the class diagram provided to you. All classes are required to be implemented while respecting every detail in the diagram. Feel free to add more classes or include additional attributes if you think it is necessary, but do not change access modifiers or the class hierarchy of the classes provided in the diagram.

IV.1 Program behaviour

Your program will take exactly one argument from the command line. This argument is the name of a text file containing the scenario that needs to be simulated. You can find an example file provided with the subject.

Executing the program will generate a file `simulation.txt` that describes the outcome of the simulation.
Example:

```
$java ro.academyplus.avaj.simulator.Simulator scenario.txt
$cat -e simulation.txt
Tower says: Balloon#B1(1) registered to weather tower.
Tower says: JetPlane#J1(2) registered to weather tower.
Tower says: Helicopter#H1(3) registered to weather tower.
Tower says: Helicopter#H4(4) registered to weather tower.
Balloon#B1(1): Let's enjoy the good weather and take some pics.
JetPlane#J1(2): It's raining. Better watch out for lightings.
Helicopter#H1(3): This is hot.
Helicopter#H4(4): My rotor is going to freeze!
Balloon#B1(1): Damn you rain! You messed up my balloon.
JetPlane#J1(2): OMG! Winter is coming!
Helicopter#H1(3): This is hot.
Helicopter#H4(4): My rotor is going to freeze!
Balloon#B1(1): It's snowing. We're gonna crash.
JetPlane#J1(2): It's raining. Better watch out for lightings.
Helicopter#H1(3): This is hot.
Helicopter#H4(4): My rotor is going to freeze!
Balloon#B1(1): Damn you rain! You messed up my balloon.
Balloon#B1(1) landing.
Tower says: Balloon#B1(1) unregistered from weather tower.
JetPlane#J1(2): OMG! Winter is coming!
Helicopter#H1(3): This is hot.
Helicopter#H4(4): My rotor is going to freeze!
```

IV.2 Scenario file

The first line of the file contains a positive integer. This number represents how many times the simulation runs. In our case, this will be the number of times a weather change is triggered.
Each subsequent line describes an aircraft that will be part of the simulation, using this format: `TYPE NAME LONGITUDE LATITUDE HEIGHT`.

IV.3 Weather generation

There are four types of weather:

- RAIN
- FOG
- SUN
- SNOW

Each 3-dimensional point has its own weather. Feel free to use whatever generation algorithm you want, as long as it takes the point's coordinates into account.

IV.4 Aircraft

- JetPlane:
  - `SUN` - Latitude increases by 10, Height increases by 2
  - `RAIN` - Latitude increases by 5
  - `FOG` - Latitude increases by 1
  - `SNOW` - Height decreases by 7
- Helicopter:
  - `SUN` - Longitude increases by 10, Height increases by 2
  - `RAIN` - Longitude increases by 5
  - `FOG` - Longitude increases by 1
  - `SNOW` - Height decreases by 12
- Balloon:
  - `SUN` - Longitude increases by 2, Height increases by 4
  - `RAIN` - Height decreases by 5
  - `FOG` - Height decreases by 3
  - `SNOW` - Height decreases by 15

IV.5 Simulation

- Coordinates are positive numbers.
- Height is in the 0-100 range.
- If an aircraft exceeds the upper height limit, it remains at 100.
- Each time an aircraft is created, it receives a unique ID. There cannot be two aircraft with the same ID.
- If an aircraft reaches height 0 or would go below it, the aircraft lands, unregisters from the weather tower, and logs a message.
- When a weather change occurs, each aircraft type needs to log a message, as seen in the example. The message format is: `TYPE#NAME(UNIQUE_ID): SPECIFIC_MESSAGE`. A funny message is appreciated during evaluation.
- Each time an aircraft registers with or unregisters from the weather tower, a message will be logged.

IV.6 Validation

The input file needs to be validated. Any abnormal behaviour due to invalid input data is not acceptable. If the input data is invalid, the program must stop execution. Any error messages will be printed to standard output.

Chapter V

Bonus part

Bonus points will be given if:

- You create your own custom exceptions to handle abnormal behaviour.

The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been completed in full and works without malfunctioning. If you have not passed ALL the mandatory requirements, your bonus part will not be evaluated at all.

Chapter VI

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Do not hesitate to double-check the names of your folders and files to ensure they are correct.